02

MGU BCA - 3RD SEM

Database Management Systems

Module 2 - ER and Relational Model, SQL

By Aben Sojan

Exam - 6 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

MGU BCA Honours DBMS (MG3CCRBCA201): 70 marks, 2 hours. Part A: 5 x 2 marks, short questions from all modules. Part B: answer any 5 of 6-mark questions. Part C: answer any 2 of 15-mark essays. Old CBCS papers (2019-2021): 80 marks, 3 hours, any-10 x 2 + any-6 x 5 + any-2 x 15. Part C hotspots: COMPANY ER diagram, Normal Forms essay, anomalies.

TOPIC 1

ER Model: Entities, Attributes, Relationships

1

ER (Entity-Relationship) model designs a database using entities, attributes and relationships.

Entity: a real-world object like Student or Book. Entity set: all similar entities.

Attribute: a property of an entity, like name or roll number.

Simple attribute: cannot be split, e.g. age. Composite: can be split, e.g. name into first and last.

Multi-valued: can have many values, e.g. phone numbers. Derived: computed from another, e.g. age from dob.

Relationship: a link between entities, e.g. Student enrolls in Course.

QUESTION BANK

1 What is an ER model? Explain entities, attributes and relationships. (Dec 2021)

IMPORTANT

The ER (Entity-Relationship) model is a high-level conceptual design tool used to describe the data needed by an organisation and how that data is connected, before any tables are created.

An entity is a real-world object that we want to store data about, such as a student, a course or an employee.

An entity set is a collection of similar entities, for example the set of all students in a college.

An attribute is a property that describes an entity, such as name, roll number or marks of a student.

Each attribute has a value for every entity; the values taken together describe that entity completely.

A relationship is an association between two or more entities, for example a student 'enrolls in' a course.

Relationships have degree: a relationship between two entity sets is binary, between three is ternary.

Relationships can also carry their own attributes, called descriptive attributes, like the date of enrolment.

Example: Student (rollno, name) and Course (code, title) with the relationship Enrolls between them gives the design 'students enrol in courses'.

2 Explain the types of attributes with examples. (Dec 2021)

IMPORTANT

Attributes describe entities, and they are classified into types based on how their values behave.

A simple attribute cannot be divided further into meaningful parts, for example age or date of birth.

A composite attribute is made of smaller parts that are meaningful on their own, for example name split into firstname and lastname, or address split into city, street and pincode.

A single-valued attribute holds at most one value per entity, for example roll number.

A multi-valued attribute can hold many values for one entity, for example a student with two phone numbers; it is shown with a double ellipse in an ER diagram.

A derived attribute is calculated from another stored attribute instead of being stored directly, for example age derived from date of birth, or total marks derived from subject marks.

A key attribute uniquely identifies each entity in a set, for example rollno for students; it is shown with an underline.

A null-valued attribute may be missing for some entities, for example email, when a student has not given one.

Example summary: a Student entity can have simple name parts, multi-valued phone numbers, derived age, and key rollno.

3 What is an entity in the ER model? (Oct 2019)

IMPORTANT

An entity is a real-world object or concept about which data is stored, like a student, a book or a department.

Each entity has attributes: properties describing it. For example, a student entity has rollno, name and marks.

An entity type (like STUDENT) is the category; an entity instance is one member, e.g. the student Aben with rollno 101.

A group of similar entities forms an entity set, e.g. all students of the college.

In an ER diagram an entity type is drawn as a rectangle, with its key attribute underlined.

Getting entities right is the first step of database design - everything else (relationships, tables, keys) builds on it.

4 Compare entity type and entity set. (Mar 2021)

IMPORTANT

Entity type is the schema-level definition: the name plus its attributes, e.g. STUDENT(rollno, name, marks).

Entity set is the actual collection of entity instances of that type at a moment, e.g. the 60 student rows currently stored.

Type is like a class in programming; the set is like the objects created from it.

The type rarely changes, while the set changes every time a row is inserted or deleted.

Example: EMPLOYEE is the entity type; the 200 employee records in the table form the entity set.

Examiners love this distinction because confusing the schema with the data is the most common beginner mistake.

5 Explain relationship attributes with an example. (Mar 2021)

IMPORTANT

A relationship attribute is a property that belongs to the relationship itself, not to either participating entity.

It is needed when the fact only makes sense for the combination. Example: Hours in WORKS_ON.

Employee E1 works on project P1 for 10 hours and on P2 for 20 hours - hours cannot sit on EMPLOYEE or PROJECT alone; it belongs to the (E1, P1) pair.

Another example: StartDate on MANAGES - the date a manager took charge of a department describes the manages-relationship.

In ER diagrams, relationship attributes are drawn as ovals attached to the diamond.

Spotting relationship attributes correctly is what separates a correct ER diagram from a wrong one.

6 **When we think of relationships as attributes, what are the value sets of these attributes? What class of data model is based on this concept? (Oct 2019)**

IMPORTANT

If a relationship is viewed as an attribute of an entity, its value is a set of references to the related entities.

Example: the works-for relationship seen from EMPLOYEE is an attribute whose value is the department entity it points to.

So the value set of such an attribute is a set of entity references (object identifiers), not simple values.

The class of data model based on this idea is the object-oriented / object data model (and EER), where relationships are stored as reference-valued attributes.

This is why object databases navigate via stored references instead of joins.

This question tests whether you understand the idea behind object-oriented databases, not just ER notation.

TOPIC 2

ER Diagrams, Key Constraints, Weak Entity Sets

2

ER diagram symbols: rectangle for entity, ellipse for attribute, diamond for relationship, line for links.

Key attribute is underlined in the diagram, e.g. rollno.

Key constraint: an attribute or set that uniquely identifies an entity.

Weak entity set: cannot be identified by its own attributes alone, e.g. Dependent of an Employee.

A weak entity depends on a strong (owner) entity through an identifying relationship.

Weak entities use a double rectangle and double diamond in the diagram.

QUESTION BANK

7 **Construct an ER diagram for the COMPANY database with departments, projects, employees and dependants. (Oct 2019, Mar 2021)**

SURE MARK

Step 1 - entity types: DEPARTMENT, PROJECT, EMPLOYEE, and DEPENDENT (a weak entity).

DEPARTMENT attributes: Dname, Dnumber (key), locations (multi-valued, shown with a double oval).

PROJECT attributes: Pname, Pnumber (key), Plocation; each project is controlled by one department.

EMPLOYEE attributes: Fname, Minit, Lname, Ssn (key), Address, Salary, Sex, Bdate.

Step 2 - relationships: MANAGES is 1:1 between DEPARTMENT and EMPLOYEE, with attribute Mgr_start_date.

CONTROLS is 1:N from DEPARTMENT to PROJECT; WORKS_FOR is N:1, every employee works for exactly one department (total participation on the employee side).

WORKS_ON is M:N between EMPLOYEE and PROJECT with the relationship attribute Hours (hours per week).

SUPERVISION is a 1:N recursive relationship: one employee supervises many, each has at most one direct supervisor.

DEPENDENT is weak: attributes Dependent_name (partial key, dashed underline), Sex, Bdate, Relationship; it depends on EMPLOYEE through the identifying relationship DEPENDENTS_OF (double diamond, double rectangle, total participation).

Draw rectangles for entities, diamonds for relationships, ovals for attributes; underline key attributes.

This exact question repeated verbatim in the Oct 2019 and Mar 2021 Part C - learn the spec cold.

8 **Explain an ER diagram with the symbols used. (Model paper)**

REPEATED

An ER diagram is a pictorial representation of the ER model, drawn with fixed symbols so that the database design can be seen at a glance.

A rectangle represents an entity set, for example a rectangle labelled Student.

An ellipse represents an attribute, for example an ellipse labelled name attached to the Student rectangle.

An underlined attribute (or underlined ellipse) is a key attribute, for example rollno underlined

under Student.

A diamond represents a relationship set between entities, for example a diamond labelled Enrolls between Student and Course.

Straight lines connect an entity set to its relationship diamond and to its attribute ellipses.

A double ellipse marks a multi-valued attribute, and a double rectangle marks a weak entity set.

A double diamond marks an identifying relationship used by a weak entity set.

Cardinality constraints are written near the lines, such as 1 or N, showing how many entities of one side connect to the other.

Example: draw Student (rectangle) with name (ellipse) and rollno (underlined), Course (rectangle) with code (underlined), and connect both through the Enrolls (diamond).

9 What is a weak entity set? Give an example.

IMPORTANT

A weak entity set is an entity set that does not have a key attribute of its own, so its entities cannot be identified without help.

It depends on a strong entity set called the owner (or identifying) entity through a relationship called the identifying relationship.

A weak entity is identified by the key of its owner plus its own partial key (discriminator), which is only unique within one owner.

Example: Dependent (name, age) is weak because two employees can both have dependents named Raju; only Employee ID plus name makes a dependent unique.

In an ER diagram a weak entity set is drawn with a double rectangle, and its identifying relationship with a double diamond.

The discriminator attribute is drawn with a dashed underline.

The relationship from a weak entity to its owner is always many-to-one and mandatory: every weak entity must be linked to an owner.

Weak entities are converted to tables later by adding the owner's primary key to them, making a combined key.

10 **Explain cardinality and participation constraints in ER diagrams. (Dec 2021)**

IMPORTANT

Cardinality ratio says how many entities on one side can relate to how many on the other: 1:1, 1:N or M:N.

Example: DEPARTMENT-EMPLOYEE works-for is 1:N (one department, many employees); STUDENT-COURSE enrolls is M:N.

Participation says whether every entity must take part: total (double line) or partial (single line).

Example: every EMPLOYEE must work for a DEPARTMENT (total participation of EMPLOYEE), but a DEPARTMENT may have zero projects (partial).

Together they are called structural constraints, and they decide the final table design and foreign keys.

These two constraints together are called structural constraints, and they directly decide your foreign keys.

TOPIC 3

Relational Model and Relational Schema

3

The relational model stores data in tables called relations.

Each table has rows (tuples) and columns (attributes).

Relational schema is the structure of the tables, e.g. Student(rollno, name, marks).

Degree: number of columns in a table. Cardinality: number of rows.

Each cell holds a single atomic value.

QUESTION BANK

11 **Explain the relational model with an example.**

IMPORTANT

The relational model stores data in two-dimensional tables called relations, where each table represents one entity set or one relationship.

A relation has a fixed set of columns called attributes, and each column has a name and a data type.

The rows of a table are called tuples, and each tuple stores the data of one entity, like one student.

Values must be atomic (indivisible), so no cell holds a list of values.

Each relation has a primary key: an attribute or set of attributes that is unique and never null,

such as rollno for Student.

A foreign key is an attribute in one table that refers to the primary key of another table, linking the two tables.

Example: Student(rollno, name, marks) with rollno as primary key; Enrolment(rollno, code) with rollno as a foreign key pointing to Student.

A relational schema is the design of these tables: the table names, their attributes, the keys and the foreign key links.

The relational model is simple, flexible and is the basis of almost every commercial DBMS like MySQL and Oracle.

12 What are degree and cardinality of a relation?

IMPORTANT

Degree of a relation is the number of attributes (columns) in the relation.

Cardinality of a relation is the number of tuples (rows) currently stored in it.

Degree is part of the schema and stays fixed, while cardinality keeps changing as rows are inserted or deleted.

Example: Student(rollno, name, marks) has degree 3 because it has three attributes.

If the college admits 60 students, the cardinality of that relation is 60.

When one new student joins, the cardinality becomes 61 but the degree remains 3.

Both terms describe the shape of the relation: degree is its width and cardinality is its height.

13 Compare relation, relationship type, relationship set and structural constraints. (Dec 2021)

IMPORTANT

Relation: a table in the relational model - rows (tuples) and columns (attributes), e.g. the STUDENT table.

Relationship type: an ER-level association between entity types, e.g. WORKS_FOR between EMPLOYEE and DEPARTMENT.

Relationship set: the actual pairs linked right now, e.g. the 200 (employee, department) pairs stored.

Structural constraints: the cardinality plus participation rules on a relationship type, e.g. 1:N with total participation.

In short: relationship type and set live in the ER design; the relation is the table you finally create; structural constraints are the rules shaping both.

This was a 15-mark Part C question - the examiner expects all four terms defined and contrasted, not just listed.

14 Define domains and tuples. (Dec 2021)

IMPORTANT

A domain is the set of allowed values for an attribute. For example, the domain of Age is integers from 0 to 150.

It works like a data type plus a permitted range, and it keeps invalid values out.

A tuple is one row of a relation: an ordered set of values, one from each attribute's domain.

Example: (101, 'Aben', 20, 85) is a tuple of STUDENT; the 20 comes from the Age domain.

So domains define what values are legal; tuples are the actual stored rows built from them.

Domains and tuples are the two building blocks of the relational model - every table is a set of tuples over domains.

15 What are the characteristics of relations? (Mar 2021)

IMPORTANT

Each relation (table) has a unique name, different from every other table.

Each attribute (column) has a unique name within the table.

All values in one column come from the same domain and are atomic (single-valued).

Each tuple (row) is unique - no two rows are fully identical.

The order of rows and of columns does not matter; the table means the same rearranged.

These rules are what make a relation stricter than an ordinary file or spreadsheet table.

TOPIC 4

Relational Algebra

4

Selection: picks rows that satisfy a condition.

Projection: picks columns and removes duplicate rows.

Set operations: union, intersection and difference combine two tables.

Joins: combine related rows from two tables into one.

Division: used for "for all" type queries.

16 Explain selection, projection and join in relational algebra.

IMPORTANT

Relational algebra is a procedural query language: queries are written as step-by-step operations on relations, each producing a new relation.

Selection picks the tuples (rows) that satisfy a given condition and keeps all attributes.

It is written with the sigma symbol, for example sigma with condition marks > 80 applied to Student returns only the students scoring above 80.

Projection picks the attributes (columns) wanted and drops the rest, automatically removing duplicate rows.

It is written with the pi symbol, for example pi with attribute name applied to Student returns just the list of student names.

Join combines tuples from two relations using a common attribute, for example Student joined with Marks on rollno.

The most common join is the natural join, which matches rows automatically on same-named columns.

Join is used whenever data needed for an answer is split across two tables, like names in Student and marks in Marks.

Example idea: to get the names of students scoring above 80, first select marks > 80 from Marks, then join with Student, then project name.

17 What are the set operations in relational algebra?

IMPORTANT

Relational algebra supports set operations that treat relations like mathematical sets of tuples.

Union combines all tuples appearing in either of the two relations, removing duplicates.

Intersection keeps only the tuples that appear in both relations.

Difference keeps the tuples that are in the first relation but not in the second.

Set operations can only be applied to union-compatible relations: both must have the same number of attributes with matching data types.

Example: if Pass has the roll numbers of students who passed and Distinction has the roll numbers with distinction, Pass UNION Distinction gives every passing roll number once.

Pass INTERSECT Distinction gives the distinction holders, and Pass MINUS Distinction gives those who passed without distinction.

Union is useful for combining results, intersection for common data, and difference for 'in A but not in B' questions.

TOPIC 5

Relational Calculus

5

Tuple relational calculus: queries use tuple variables to describe the wanted rows.

Domain relational calculus: uses domain (column value) variables instead of tuples.

Both are non-procedural: you say what you want, not how to get it.

SQL is largely based on tuple relational calculus.

QUESTION BANK

18 **Distinguish between tuple and domain relational calculus.**

IMPORTANT

Relational calculus is a non-procedural query language: you describe what result you want, and the DBMS decides how to fetch it.

In tuple relational calculus, the variables range over whole tuples (rows) of a relation.

A typical tuple calculus query says 'find a tuple t such that t belongs to Student and t.marks > 80'.

In domain relational calculus, the variables range over individual attribute values (domains), not whole rows.

A typical domain calculus query says 'find a name value n such that there is a Student row with name n and marks > 80'.

Tuple calculus is closer to SQL row-wise thinking, while domain calculus matches SQL's column-wise SELECT list more directly.

Both are declarative and equally expressive: anything written in one can be written in the other.

Example idea: the question 'names of students with marks above 80' is expressed in both as a condition on the Student relation, with no steps mentioned.

19 Why is relational calculus called non-procedural?

IMPORTANT

Relational calculus is called non-procedural because the query only declares the desired result, never the steps to obtain it.

You state the condition the answer must satisfy, like 'marks greater than 80', without saying how to scan or index the table.

The DBMS query optimiser then works out the fastest execution plan on its own.

Relational algebra is the opposite: it is procedural, giving the exact sequence of selection, projection and join operations.

SQL follows the calculus idea: you write SELECT name FROM Student WHERE marks > 80 and let the engine plan the execution.

This makes queries shorter, easier to write and easier for the DBMS to optimise.

So 'non-procedural' simply means 'say what you want, not how to get it'.

TOPIC 6

SQL: DDL and DML

6

DDL (Data Definition Language): CREATE, ALTER, DROP define or change structure.

DML (Data Manipulation Language): INSERT, UPDATE, DELETE, SELECT handle data.

DCL: GRANT and REVOKE manage permissions. TCL: COMMIT and ROLLBACK manage transactions.

QUESTION BANK

20 Distinguish between DDL and DML with examples.

IMPORTANT

SQL commands are grouped by purpose: DDL defines and changes the structure of the database, while DML works with the data stored inside it.

DDL (Data Definition Language) includes CREATE TABLE, ALTER TABLE, DROP TABLE and TRUNCATE TABLE.

DDL commands change the schema itself, like adding a table or a column, and they usually commit automatically.

DML (Data Manipulation Language) includes INSERT, UPDATE, DELETE and SELECT.

DML commands change or read the rows of a table, and their changes can be rolled back with ROLLBACK before they are committed.

Example DDL: CREATE TABLE Student (rollno INT PRIMARY KEY, name VARCHAR(50)); creates the structure of the table.

Example DML: INSERT INTO Student VALUES (1, 'Aben', 85); adds one row of data to that table.

In short: DDL builds and modifies the container, DML fills, updates and reads what is inside it.

21 Write DDL commands to create a Student table and then add a column to it.

IMPORTANT

DDL is used to define the structure of the table, starting with the CREATE TABLE command.

The Student table is created with rollno as an integer primary key, name as a required text field, and marks as an integer.

The PRIMARY KEY constraint keeps rollno unique and non-null, and NOT NULL forces every row to have a name.

Later, when an email address is needed, ALTER TABLE adds the new column without touching the existing rows.

Code lines (unchanged):

Create the table with a primary key:

```
CREATE TABLE Student (
rollno INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
marks INT
);
```

Add a new column later:

```
ALTER TABLE Student ADD email VARCHAR(60);
```

Explanation: CREATE TABLE defines the new relation and its constraints in one step.

ALTER TABLE with ADD appends the email column to the existing table, and old rows simply get a null email until updated.

22 What is SQL? (Oct 2019)

IMPORTANT

SQL (Structured Query Language) is the standard language for working with relational databases.

It is used to create tables, insert and update data, and ask questions (queries) of the database.

It has sub-languages: DDL (CREATE, ALTER, DROP), DML (SELECT, INSERT, UPDATE, DELETE), DCL and TCL.

Example: SELECT name FROM Student WHERE marks > 80; lists the names of high scorers.

Almost every relational DBMS - MySQL, Oracle, PostgreSQL - understands SQL, so the skill transfers between systems.

SQL is the one language every database job expects, which is why it opens the practical module too.

23 What are the basic DDL commands in SQL? (Dec 2021)

IMPORTANT

DDL (Data Definition Language) commands define or change the structure of tables.

CREATE makes a new table:

```
CREATE TABLE Student(rollno INT PRIMARY KEY, name VARCHAR(30));
```

ALTER changes an existing table:

```
ALTER TABLE Student ADD COLUMN age INT;
```

DROP deletes a table completely:

```
DROP TABLE Student;
```

TRUNCATE removes all rows but keeps the structure:

```
TRUNCATE TABLE Student;
```

DDL commands take effect immediately and cannot be rolled back in most systems.

24 Write the syntax of the SELECT command with an example. (Dec 2021)

IMPORTANT

Syntax:

```
SELECT column1, column2 FROM table_name WHERE condition;
```

SELECT * picks all columns; WHERE filters rows.

Example:

```
SELECT name, marks FROM Student WHERE marks > 80;
```

This returns only the name and marks columns of students scoring above 80.

Example with all columns:

```
SELECT * FROM Student;
```

This lists the full table.

25 Give the general form of a SQL query. (Mar 2021)

IMPORTANT

General form:

```
SELECT <attribute list>
FROM <table list>
WHERE <condition>
GROUP BY <grouping attributes>
HAVING <group condition>
ORDER BY <attribute list>;
```

SELECT and FROM are compulsory; the rest are optional.

The engine filters rows (WHERE), groups them (GROUP BY), filters groups (HAVING), then sorts (ORDER BY).

Example:

```
SELECT dept, AVG(marks) FROM Student GROUP BY dept HAVING AVG(marks) > 70;
```

26 **Describe the four clauses in the syntax of a simple SQL retrieval query. (Oct 2019)**

IMPORTANT

SELECT: lists which columns to show, e.g. SELECT name, marks.

FROM: names the table (or tables) to read, e.g. FROM Student.

WHERE: filters individual rows before any grouping, e.g. WHERE marks > 80.

ORDER BY: sorts the final result, e.g. ORDER BY marks DESC.

Together:

```
SELECT name, marks FROM Student WHERE marks > 80 ORDER BY marks DESC;
```

This shows high scorers with the best first.

27 **Consider the table Student(RollNo, Name, Age, Marks). Write SQL queries to: (a) add a new column Percentage, (b) find students whose names start with 'A', (c) find students aged between 18 and 22, (d) find the minimum and maximum marks. (Model paper)**

IMPORTANT

(a) Add the column with ALTER TABLE:

```
ALTER TABLE Student ADD COLUMN Percentage FLOAT;
```

(b) Names starting with A, using LIKE:

```
SELECT * FROM Student WHERE Name LIKE 'A%';
```

LIKE with 'A%' matches names starting with A; % means any following characters.

(c) Age range with BETWEEN:

```
SELECT * FROM Student WHERE Age BETWEEN 18 AND 22;
```

BETWEEN checks an inclusive range, the same as Age >= 18 AND Age <= 22.

(d) Extremes with aggregate functions:

```
SELECT MIN(Marks), MAX(Marks) FROM Student;
```

MIN and MAX return the smallest and largest marks in the table.

TOPIC 7

Aggregate Functions and Logical Operators

7

COUNT: number of values. SUM: total. AVG: average. MIN and MAX: smallest and largest.

Aggregate functions ignore null values except COUNT(*).

Logical operators AND, OR, NOT combine conditions in WHERE.

QUESTION BANK

28 Explain aggregate functions in SQL with examples. (Mar 2021)

IMPORTANT

Aggregate functions in SQL take many rows as input and return a single summary value.

COUNT(*) counts the number of rows in a table, including rows with nulls.

SUM(marks) adds up all the values of a numeric column, and AVG(marks) gives their average.

MIN(marks) returns the smallest value and MAX(marks) returns the largest value in the column.

Aggregates ignore null values except COUNT(*), so they work on the real data only.

Example query (unchanged):

```
SELECT COUNT(*), AVG(marks), MAX(marks) FROM Student;
```

Explanation: COUNT(*) tells how many students are in the table, AVG(marks) gives the class average, and MAX(marks) shows the top score.

The FROM clause picks the Student table, and all three aggregates are computed over its full set of rows in one pass.

Aggregates are the standard way to answer 'how many', 'average', 'total' and 'maximum' questions in exams.

29 Write a query to find the number of students with marks above 80.

IMPORTANT

To count only the students above a cut-off, COUNT(*) is combined with a WHERE condition.

The WHERE clause filters rows first, keeping only those with marks above 80.

COUNT(*) then counts exactly those filtered rows, so students with 80 or less are not counted.

Code line (unchanged):

Use COUNT with a WHERE condition:

```
SELECT COUNT(*) FROM Student WHERE marks > 80;
```

Explanation: FROM Student picks the table, WHERE marks > 80 keeps only the rows scoring above 80, and COUNT(*) returns how many such rows exist.

The condition uses the greater-than comparison, so a student with exactly 80 is excluded.

The result is a single number, for example 12, which is the count asked for.

TOPIC 8

8

Predicates and Clauses

LIKE: pattern matching; % matches many characters, _ matches one.

BETWEEN: value inside a range. IN: value inside a list.

DISTINCT: removes duplicate rows. AS (alias): renames a column or table.

GROUP BY: groups rows with the same value. HAVING: filters groups.

ORDER BY: sorts results. LIMIT: restricts how many rows are returned.

QUESTION BANK

30 **Explain GROUP BY and HAVING with an example query. (Dec 2021, Oct 2019)**

REPEATED

GROUP BY collects rows that share the same value in a column into separate groups, so aggregates can be computed per group.

HAVING filters these groups, in the same way that WHERE filters individual rows.

WHERE cannot be used with aggregate functions, but HAVING is designed exactly for conditions on aggregates.

The order of execution is: WHERE filters rows first, GROUP BY forms groups, HAVING filters the groups.

Example: to find departments with more than 5 students, rows are grouped by dept and groups with a small count are removed.

Code lines (unchanged):

GROUP BY collects rows with the same value into groups.

HAVING filters those groups, like WHERE filters rows.

Example: departments having more than 5 students:

```
SELECT dept, COUNT(*) FROM Student
GROUP BY dept HAVING COUNT(*) > 5;
```

Explanation: GROUP BY dept makes one group per department, COUNT(*) counts students in each group, and HAVING COUNT(*) > 5 keeps only the large departments.

The SELECT list shows the department name with its student count.

31 Distinguish between WHERE and HAVING.

IMPORTANT

WHERE and HAVING both filter data, but they work at different stages of a query.

WHERE filters individual rows before any grouping happens.

HAVING filters groups after GROUP BY has been applied.

WHERE cannot use aggregate functions like COUNT or AVG, because aggregates are computed after WHERE runs.

HAVING is meant for conditions on aggregates, like COUNT(*) > 5.

A query can use both: WHERE marks > 40 keeps only the passing rows, then GROUP BY forms groups, then HAVING COUNT(*) > 5 drops the small groups.

Example: WHERE marks > 40 picks the individual rows; HAVING COUNT(*) > 5 picks the qualifying groups.

So the rule is: filter rows with WHERE, filter groups with HAVING.

32 What is the use of LIKE and DISTINCT? Give examples.

IMPORTANT

LIKE is used in the WHERE clause to match text against a pattern.

The percent sign % stands for any number of characters, and the underscore _ stands for exactly one character.

So LIKE 'A%' matches every name starting with A, like Aben or Ansu.

Code line (unchanged):

```
SELECT * FROM Student WHERE name LIKE 'A%';
```

Explanation: WHERE name LIKE 'A%' keeps only the rows whose name begins with the letter A, and SELECT * returns all their columns.

DISTINCT removes duplicate values from the result, so each value appears only once.

Code line (unchanged):

```
SELECT DISTINCT dept FROM Student;
```

Explanation: without DISTINCT, one dept value would repeat for every student in it; DISTINCT collapses them to a single list of department names.

LIKE is for pattern searching and DISTINCT is for removing duplicates.

33 How can you order rows in SQL? (Dec 2021)

IMPORTANT

Rows are ordered with the ORDER BY clause at the end of a SELECT query.

Syntax:

```
SELECT columns FROM table ORDER BY column ASC | DESC;
```

ASC (ascending) is the default; DESC gives highest-first order.

Example:

```
SELECT name, marks FROM Student ORDER BY marks DESC;
```

This lists toppers first.

Multiple columns are allowed:

```
SELECT name, dept, marks FROM Student ORDER BY dept ASC, marks DESC;
```

This sorts by department, then by marks within each department.

TOPIC 9

Joins in SQL

9

INNER JOIN: only rows that match in both tables.

LEFT OUTER JOIN: all rows from the left table plus matches from the right.

RIGHT OUTER JOIN: all rows from the right table plus matches from the left.

FULL OUTER JOIN: all rows from both tables, nulls where there is no match.

NATURAL JOIN: joins automatically on all same-named columns.

EQUI JOIN: a join written with an equality condition.

QUESTION BANK

34 Explain the different types of joins with examples. (Model paper)

IMPORTANT

A join combines rows from two tables using a related column, usually a primary key matching a foreign key.

INNER JOIN returns only the rows that have a match in both tables; unmatched rows from either side are dropped.

LEFT JOIN (left outer join) returns every row from the left table, filling right-side columns with null when there is no match.

RIGHT JOIN returns every row from the right table, filling left-side columns with null when there is no match.

FULL JOIN returns everything from both tables, with nulls filling whichever side has no match.

NATURAL JOIN joins automatically on all columns with the same name in both tables, like rollno.

Example with Student and Marks on rollno: INNER JOIN shows students who have marks rows; LEFT JOIN shows all students even those without marks.

The choice of join depends on the question: use INNER JOIN for 'only matching data' and LEFT JOIN for 'all from one side, data where available'.

35 Distinguish between INNER JOIN and LEFT OUTER JOIN.

IMPORTANT

INNER JOIN and LEFT OUTER JOIN differ in how they treat rows that have no match in the other table.

INNER JOIN returns only the rows where a matching row exists in both tables.

LEFT OUTER JOIN returns every row from the left table, whether or not a match exists.

For a left-table row with no match, the columns from the right table appear as null.

INNER JOIN therefore shrinks the result to matched pairs only, while LEFT JOIN keeps the full left side.

Example: joining Student with Marks, INNER JOIN hides students who have no marks rows, while LEFT JOIN still lists them with null marks.

So INNER JOIN answers 'which students have marks', and LEFT JOIN answers 'all students and their marks if any'.

36 Write a query using INNER JOIN to show student names with their marks.

IMPORTANT

To show each student's name together with their marks, the Student and Marks tables are joined on the common rollno column.

INNER JOIN keeps only the students who actually have marks rows, so the result has no unmatched entries.

The ON clause states the matching condition: Student.rollno = Marks.rollno.

Qualified column names like Student.name avoid confusion when both tables share column names.

Code lines (unchanged):

Join Student and Marks on the common rollno column:

```
SELECT Student.name, Marks.subject, Marks.mark
```

```
FROM Student INNER JOIN Marks
ON Student.rollno = Marks.rollno;
```

Explanation: FROM with INNER JOIN pairs each student row with its marks rows where the roll numbers are equal, and SELECT picks the name, subject and mark.

The ON clause performs the match, and the SELECT list decides which columns appear in the final answer.

37 Write a query using LEFT JOIN to list all students even if they have no marks.

IMPORTANT

LEFT JOIN is used when every student must appear in the result even if the Marks table has no row for them.

Unlike INNER JOIN, it never drops a left-table row: unmatched students are shown with null marks.

The ON clause still gives the matching condition Student.rollno = Marks.rollno.

This is the correct join for questions like 'list all students, with marks where available'.

Code lines (unchanged):

LEFT JOIN keeps every student row:

```
SELECT Student.name, Marks.mark
FROM Student LEFT JOIN Marks
ON Student.rollno = Marks.rollno;
```

Explanation: FROM Student LEFT JOIN Marks starts from all student rows and attaches marks rows where roll numbers match; missing marks show as null.

SELECT picks just the name and the mark, giving one row per student with a blank mark where no marks row exists.

38 What is the use of the INTERSECT operator? (Oct 2019)

IMPORTANT

INTERSECT returns only the rows common to two query results (set intersection).

Both queries must return the same number of columns with compatible types.

Example:

```
SELECT rollno FROM CricketTeam INTERSECT SELECT rollno FROM FootballTeam;
```

This gives the roll numbers of students who are in both teams.

Duplicates are removed automatically; only distinct common rows appear.

39 Explain UNIQUE and EXISTS in SQL. (Dec 2021)

IMPORTANT

UNIQUE is a constraint: it forbids duplicate values in a column but allows NULL.

Example:

```
CREATE TABLE Student(rollno INT PRIMARY KEY, email VARCHAR(50) UNIQUE);
```

It is weaker than PRIMARY KEY (which also forbids NULL) and suits alternate keys like email or phone.

EXISTS is a subquery test: WHERE EXISTS (subquery) is true when the subquery returns at least one row.

Example:

```
SELECT name FROM Student S WHERE EXISTS (SELECT * FROM Marks M WHERE M.rollno = S.rollno AND M.mark > 90);
```

So UNIQUE guards data quality at table-design time, while EXISTS filters rows at query time.